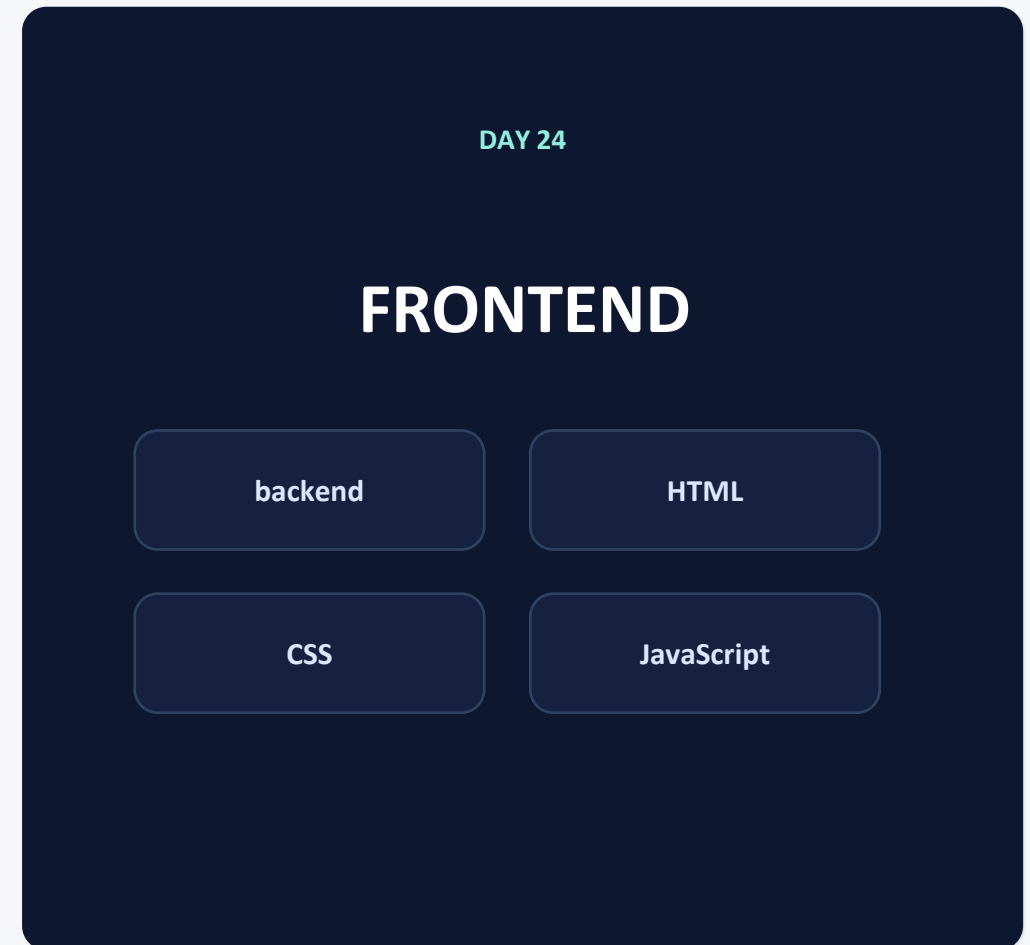


Frontend, backend и полный путь данных

Проследить полный путь данных в веб-приложении:
браузер → frontend → HTTP → backend → база данных →
JSON → интерфейс.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Проследить полный путь данных в веб-приложении: браузер → frontend → HTTP → backend → база данных → JSON → интерфейс.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример записи и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

API, REST и JSON

Что уже есть в нашей модели.



СЕГОДНЯ

Frontend, backend и полный путь данных

Вчера: API, REST и JSON. Сегодня мы добавляем следующий слой: frontend, backend и полный путь данных.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Когда видно весь маршрут данных, легче понимать, где искать ошибку: в интерфейсе, сети, серверной логике или данных.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

АНАЛОГИЯ

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Веб-приложение похоже на магазин:
витрина — frontend, склад и правила — backend, учёт — база, а HTTP — курьер между частями.

ПЕРЕВОД В IT**frontend**

часть приложения, отвечающая за пользовательский интерфейс

backend

серверная часть с логикой, доступом и данными

HTML

язык структуры веб-страницы

Слой 1: frontend • backend • HTML

Сначала смысл, затем профессиональный термин.

**frontend**

часть приложения, отвечающая за пользовательский интерфейс

**backend**

серверная часть с логикой, доступом и данными

**HTML**

язык структуры веб-страницы

Слой 2: CSS • JavaScript • framework

Сначала смысл, затем профессиональный термин.

**CSS**

язык визуального оформления веб-страницы

**JavaScript**

язык поведения веб-интерфейса

**framework**

готовый каркас и набор инструментов разработки

Слой 3: DOM • responsive design

Сначала смысл, затем профессиональный термин.

DOM

объектное представление HTML-страницы

responsive design

адаптация интерфейса к разным экранам

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: component • build tool

Сначала смысл, затем профессиональный термин.

component

самостоятельная переиспользуемая часть интерфейса

build tool

инструмент подготовки исходного кода к запуску

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day24_step1

```
Frontend -> HTTP request -> Backend -> Database  
Frontend <- JSON response <- Backend <- Database
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

Frontend -> HTTP request -> Backend -> Database

Python выполняет эту строку сверху вниз.

Frontend <- JSON response <- Backend Database

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
Frontend -> HTTP request -> Backend -> Database  
Frontend <- JSON response <- Backend <- Database
```

РЕЗУЛЬТАТ

Один пользовательский клик
через несколько слоёв

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
click "Выполнено"  
-> PATCH /api/tasks/7  
-> UPDATE tasks SET done = true  
-> {"id": 7, "done": true}  
-> интерфейс ставит галочку
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Данные изменились в БД и состояние отразилось в интерфейсе

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

Browser -> Database напрямую

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Обычно браузер не должен напрямую обращаться к внутренней базе: backend контролирует авторизацию, бизнес-правила и доступ к данным.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

Browser -> Frontend -> API/Backend -> Database

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Разобрать кнопку «Выполнено» от клика до обновлённой строки в интерфейсе.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Обработчик frontend ловит клик.**
- 2 Отправляет PATCH запрос.
- 3 Backend проверяет пользователя и task id.
- 4 База обновляет done.
- 5 Backend возвращает JSON, frontend перерисовывает состояние.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

1. click
2. PATCH /api/tasks/7 {"done": true}
3. backend validates
4. database UPDATE

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



solution_final

1. click
2. PATCH /api/tasks/7 {"done": true}
3. backend validates
4. database UPDATE
5. 200 {"id":7,"done":true}
6. DOM updates

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

На экране задача помечена выполненной

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобрannую — но теперь первый ход за тобой.

Опиши полный путь GET /profile от браузера до отображения имени пользователя.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Запрос → route/backend → SELECT в БД → JSON/HTML → frontend.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
GET /profile  
-> backend  
-> SELECT user  
-> response  
-> render name
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Пользователь видит имя из данных, полученных через backend

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Для ошибки «галочка не сохраняется после перезагрузки» назови минимум три слоя, которые нужно проверить.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
check frontend request  
check backend response/status  
check database UPDATE/COMMIT
```

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Проблему можно локализовать по цепочке, а не гадать

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Обвинять frontend во всех визуальных симптомах.

02

Разрешать клиенту доверенно передавать критические решения.

03

Смешивать HTML/CSS/JS с серверной БД как один слой.

04

Не смотреть Network/логи при поиске ошибки.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «frontend» и какую проблему это понятие помогает описать?

2

Чем «frontend» отличается от «CSS»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 frontend — часть приложения, отвечающая за пользовательский интерфейс.

2 frontend: часть приложения, отвечающая за пользовательский интерфейс. CSS: язык визуального оформления веб-страницы.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **Один пользовательский клик проходит через несколько слоёв**

4 **Обычно браузер не должен напрямую обращаться к внутренней базе: backend контролирует авторизацию, бизнес-правила и доступ к данным.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ **Проследить полный путь данных в веб-приложении: браузер → frontend → HTTP → backend → база данных → JSON → интерфейс.**
- ✓ **Читать и объяснять пример темы «Frontend, backend и полный путь данных».**
- ✓ **Находить типичную ошибку и формулировать причину.**
- ✓ **Решать похожую задачу по шагам, а не угадывать готовый код.**

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Первое веб-приложение на Flask

Завтра: Первое веб-приложение на Flask. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.